DEPARTMENT OF INDUSTRIAL MANAGEMENT

FACULTY OF SCIENCE | UNIVERSITY OF KELANIYA

INTE 22283 – Mobile Application Development

Week 2: Mobile Operating Systems

Sammani Maddumahewage

Lecturer (Probationary)

BSc (Hons) in Information Technology



Today's Learning Outcomes

- Describe the architecture of the Android operating system (kernel, runtime, framework, application layers)
- Describe the architecture of iOS and compare it with Android
- Identify other mobile operating systems and their market relevance
- Explain the Android application lifecycle and process model
- Discuss permission models and sandboxing in mobile operating systems
- Compare update cycles, fragmentation, and device compatibility issues
- Set up the Android Studio development environment and emulator
- Explore the structure of a basic Android project

Recap: Where We Are in the Module

- **Last week:** We covered mobile computing fundamentals and how mobile networks connect devices
- **This week:** We move from the network layer up to the operating system — the software platform that every mobile app runs on top of
- **Why this matters:** Every design decision in later weeks (UI, architecture, APIs, security) depends on which OS the app targets and how that OS manages apps, permissions, and resources

What Is a Mobile Operating System?

- **Definition:** System software that manages hardware resources, provides a runtime environment for apps, and exposes APIs that developers use to build applications
- **Core responsibilities:** Process management, memory management, security enforcement, hardware abstraction, and user interface services
- **Why mobile OS design differs from desktop:** Mobile OSes must operate under strict battery, memory, and storage constraints while keeping the device responsive and secure

Android Architecture: The Software Stack

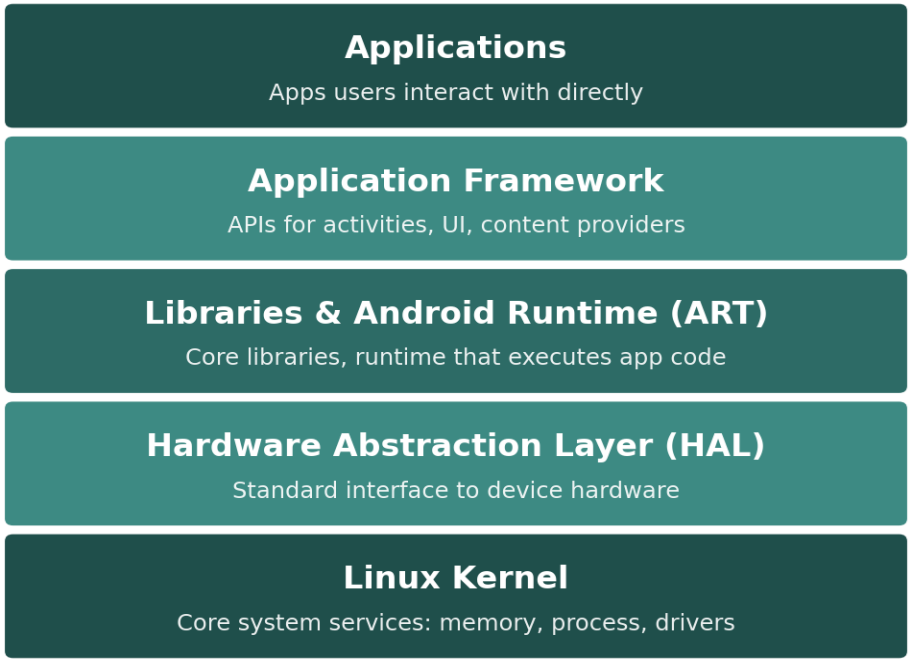


Figure 1: Android is organized into five layers, each building on the one below it

- **Applications:** The apps users actually open and interact with
- **Application Framework:** Reusable APIs (Activity Manager, Content Providers, Views) that apps are built on
- **Libraries & ART:** Native libraries plus the Android Runtime that compiles and executes app code
- **HAL & Linux Kernel:** The hardware abstraction layer and kernel that manage drivers, memory, and processes

Android Architecture Layers

- **Linux Kernel**
  - Provides core system services: process management, memory management, device drivers, and security
- **Hardware Abstraction Layer (HAL)**
  - Standard interfaces that let higher layers use hardware (camera, sensors, audio) without knowing device-specific details
- **Android Runtime (ART) and Native Libraries**
  - ART compiles app bytecode to native instructions; libraries provide functionality like SQLite, WebKit, and SSL
- **Application Framework**
  - Java/Kotlin APIs that apps use directly: Activity Manager, Window Manager, Content Providers, Notification Manager
- **Applications**
  - Both system apps (Phone, Settings) and third-party apps installed by the user, all running on the same framework

iOS Architecture Overview

- **Core OS layer:** The lowest level — kernel (XNU, based on Darwin/BSD), drivers, and core hardware services
- **Core Services layer:** Fundamental system services such as networking, location, and data management
- **Media layer:** Graphics, audio, and video technologies (Core Graphics, Core Animation, AVFoundation)
- **Cocoa Touch layer:** The topmost layer providing the UI frameworks (UIKit, SwiftUI) that apps are built with directly

Android vs. iOS: Architectural Comparison

Table 1: Key architectural and ecosystem differences between the two dominant mobile platforms

Aspect	Android	iOS
Kernel	Linux kernel	XNU (Darwin/BSD-based)
Primary languages	Kotlin, Java	Swift, Objective-C
App distribution	Google Play + sideloading	App Store only (largely)
Hardware vendors	Many (Samsung, Xiaomi, etc.)	Apple only
Customization	High (open ecosystem)	Lower (tightly controlled)

Why the Architectural Differences Matter

- **Openness vs. control:** Android's open-source, multi-vendor model enables device diversity but creates the fragmentation challenges discussed shortly
- **Single-vendor consistency:** Apple's tight hardware-software integration in iOS gives more predictable performance and update behavior, at the cost of platform flexibility
- **Developer implication:** Targeting both platforms generally requires either two native codebases or a cross-platform framework — the subject of Week 3 and Week 12

Other Mobile Operating Systems and Market Relevance

- **HarmonyOS (Huawei):** Developed after Huawei lost access to Google services; growing market share primarily in China
- **KaiOS:** A lightweight OS for feature phones, significant in emerging markets where smartphones remain costly
- **Historical context:** Platforms like Windows Phone and BlackBerry OS once held meaningful market share but have been discontinued
- **Practical takeaway:** Android and iOS together account for the overwhelming majority of the global market — this module focuses primarily on Android for hands-on development

Why Lifecycle Management Exists

- **The problem:** Mobile devices have limited memory, and users frequently switch between apps, receive calls, or lock their screens
- **The OS's job:** Android must decide which apps stay in memory, which get paused, and which get terminated — without losing user data or confusing the user
- **The developer's job:** Apps must correctly respond to lifecycle events so they save state, release resources, and resume correctly

The Activity Lifecycle

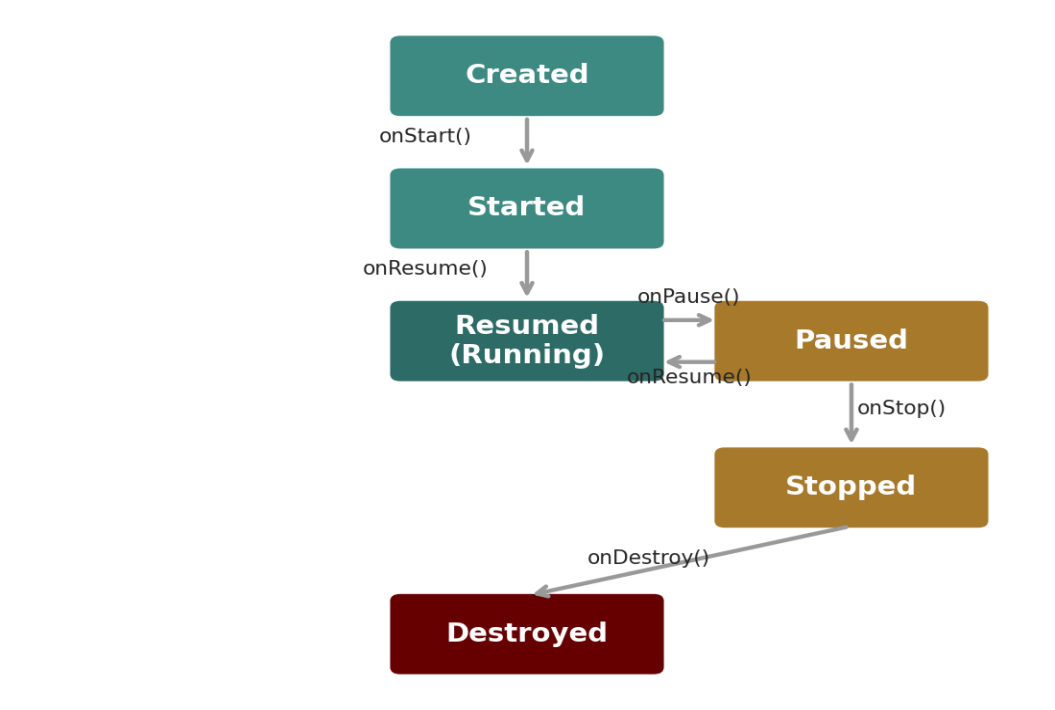


Figure 2: Every Activity moves through a well-defined sequence of states, each with a corresponding callback method

- **Created → Started → Resumed:** The activity becomes visible and interactive
- **Paused:** Activity is partially visible but not in focus (e.g., a dialog appears)
- **Stopped:** Activity is no longer visible (e.g., user switched apps)
- **Destroyed:** Activity's resources are released, either by the user or the system

Key Lifecycle Callback Methods

- **onCreate()**
  - Called once when the Activity is first created; used to initialize UI and data
- **onStart() / onResume()**
  - Activity becomes visible (onStart) and then interactive (onResume)
- **onPause()**
  - Called when the activity loses focus; used to save lightweight state and pause ongoing actions (e.g., video playback)
- **onStop()**
  - Called when the activity is no longer visible; used to release resources not needed while hidden
- **onDestroy()**
  - Final cleanup before the activity is removed from memory

The Android Process Model

- **Each app runs in its own process:** By default, every Android app runs in its own Linux process with a unique user ID, isolating it from other apps
- **Process priority levels:** Android ranks processes (foreground, visible, service, background, empty) and kills lower-priority processes first when memory is needed
- **Why this matters for developers:** An app moved to the background can be killed at any time — robust apps must persist important state so users don't lose progress
- **Multi-process apps:** Some apps deliberately run components in separate processes (e.g., isolating a crash-prone library) for stability

The Sandbox Model

- **Definition:** Each app runs in an isolated environment with restricted access to system resources and other apps' data
- **Android's implementation:** Each app is assigned a unique Linux user ID (UID); the kernel enforces that one app's files and memory cannot be accessed by another app directly
- **iOS's implementation:** Apps run in a similar sandbox enforced by the kernel, with strict entitlements controlling what system resources an app can use
- **Why sandboxing matters:** It limits the damage a malicious or buggy app can do — this is a foundational security mechanism we revisit in depth in Week 13

Permission Models: Install-Time vs. Runtime

Install-Time Permissions

- **Used for:** Low-risk permissions (e.g., internet access)
- **Behavior:** Granted automatically when the app is installed
- **User control:** Limited — user can only choose not to install the app

Runtime Permissions

- **Used for:** Sensitive permissions (camera, location, contacts)
- **Behavior:** App must request permission while running; user approves or denies
- **User control:** High — permissions can be revoked at any time in system settings

Why Modern Permission Models Evolved This Way

- **Early Android (pre-6.0):** Used install-time-only permissions — users had to accept everything or not install the app at all
- **Modern Android (6.0+) and iOS:** Use runtime permission requests for sensitive capabilities, giving users granular, revocable control
- **Principle of least privilege:** Well-designed apps should request only the permissions they actually need, when they need them — this is both a security best practice and an app-store review requirement

Android Fragmentation

- **Definition:** The wide variety of Android OS versions, screen sizes, and hardware capabilities in active use simultaneously
- **Root cause:** Android updates depend on device manufacturers and carriers to distribute them, leading to long delays or many devices never receiving updates
- **Practical consequence for developers:** Apps must be tested across multiple OS versions and screen configurations, and gracefully handle missing newer-API features on older devices
- **iOS comparison:** Apple controls hardware and software together, so iOS updates reach the vast majority of supported devices quickly, greatly reducing fragmentation

Managing Compatibility in Android Development

Table 2: Common tools developers use to handle fragmentation

Tool / Concept	Purpose
minSdkVersion	Sets the oldest Android version the app supports
targetSdkVersion	Declares the version the app is optimized and tested for
AndroidX libraries	Backward-compatible APIs that work across many OS versions
Responsive layouts	Layouts that adapt to different screen sizes and densities

Worked Example: An App Losing Data on Rotation

- **Scenario:** A user fills out a form, rotates their phone, and the form data disappears
- **What actually happened:** Screen rotation destroys and recreates the Activity by default — triggering onDestroy() and a fresh onCreate()
- **Root cause:** The developer didn't save the form data during the lifecycle transition
- **Correct fix:** Save transient UI state in onSaveInstanceState() and restore it in onCreate(), or use a lifecycle-aware ViewModel that survives configuration changes

Worked Example: A Permission Denied at Runtime

- **Scenario:** A camera app requests camera access; the user denies it
- **What the OS does:** Android does not crash the app — it simply does not grant access, and the app must handle this case explicitly
- **Poor design:** An app that crashes or freezes when permission is denied
- **Good design:** The app explains why the permission is needed and offers an alternative path (e.g., manually entering data) if permission is refused

Setting Up the Android Studio Environment

- **Android Studio:** The official IDE for Android development, built on JetBrains' IntelliJ platform
- **Android SDK:** Software Development Kit containing the libraries and tools needed to build, debug, and run Android apps
- **Android Virtual Device (AVD) / Emulator:** A simulated Android device used to test apps without physical hardware
- **Gradle:** The build system Android Studio uses to compile code, manage dependencies, and package the app

Structure of a Basic Android Project

- **manifests/AndroidManifest.xml**
  - Declares the app's components, permissions, and metadata the OS needs to run it
- **java/ (or kotlin/)**
  - Contains the source code: Activities, Fragments, and supporting classes
- **res/ (resources)**
  - Layout XML files, images, strings, and other non-code assets, organized by type
- **build.gradle (module and project level)**
  - Configures dependencies, SDK versions, and build settings
- **Gradle Scripts**
  - Supporting build configuration files generated and managed by Android Studio

CONCLUSION

Key Takeaways

- Android's architecture is a layered stack: Linux kernel, HAL, libraries/ART, application framework, and applications
- iOS uses a comparable layered model but with tighter hardware-software integration and a more controlled distribution model
- The Activity lifecycle (Created, Started, Resumed, Paused, Stopped, Destroyed) governs how apps respond to system and user events
- Sandboxing isolates each app for security; permission models (install-time vs. runtime) control access to sensitive resources
- Fragmentation is a major practical challenge in Android development, managed through SDK version targeting and compatibility libraries

Summary

- This week moved from the network layer to the operating system layer, examining how Android and iOS are architected, how apps move through a managed lifecycle, and how sandboxing and permissions protect users.
- We also looked at why fragmentation is a uniquely significant challenge for Android developers, and set up the practical foundation — Android Studio and project structure — needed for hands-on development.
- Next week, we build on this foundation by comparing native, hybrid, and cross-platform approaches to actually building mobile applications.

Discussion Questions

- Why might Apple's tightly controlled ecosystem lead to less fragmentation than Android's open ecosystem? Is this purely an advantage, or are there trade-offs?
- Consider an app that plays music. Which lifecycle callback(s) should it use to pause music when the user switches apps, and why?
- A weather app requests location permission the moment it opens, before explaining why. How might this affect user trust, and how would you redesign the request flow?
- If you were choosing between developing for Android first or iOS first for a new startup, what factors from today's lecture would influence your decision?

Thank You !

Questions?

Email: Sammani@kln.ac.lk | Department of Industrial Management